CHAPTER 11

Basic Input and Output

Ruby provides what looks at first sight like two separate sets of input/output (I/O) routines. The first is the simple interface—we've been using that one a lot so far:

```
print "Enter your name: "
name = gets
```

This whole set of I/O-related methods is implemented in the Kernel module, including gets, open, print, printf, putc, puts, readline, readlines, and test. The I/O methods are available to all objects, and make it simple and convenient to write straightforward Ruby programs. These methods typically do I/O to standard input and standard output, which makes them useful for writing simple tasks.

The other way to do IO, which gives you a lot more control, is to use Ruby's dedicated IO classes.

What Is an IO Object?

Ruby defines a single base class, IO, to handle input and output. This base class is subclassed by classes File and BasicSocket to provide more specialized behavior, but the principles are the same. An IO object is a bidirectional channel between a Ruby program and some external resource.

In this chapter, we'll be concentrating on class IO and its most commonly used subclass, class File.

Opening and Closing Files

You can create a new file object using File.new:

```
file = File.new("testfile", "r")
 # ... process the file
file.close
```

The first parameter to the method is the filename. The second is called the *mode string*, which lets you declare whether you are opening the file for reading, writing, or both. Here we opened testfile for reading with an "r". We could also have used "w" for write or "r+" for read-write.